DOCUMENT TECHNIQUE · ARCHITECTURE

Kabyle LLM Benchmark

Comment le projet est conçu, comment chaque brique fonctionne, et pourquoi elle a été faite ainsi. Écrit pour être lu sans connaissances préalables en IA.

Dépôt : github.com/hasdaouiryma16-cmd/kabyle-llm-benchmark
État décrit : 31 octobre 2026, dernier commit `b66bec4`
Autrice du projet et du jeu de test : hasdaouiryma16-cmd

Sommaire

1. **Le projet en une page** : le problème, l'objectif, les trois phases
2. **Les notions d'IA à connaître** : LLM, token, prompt, API, few-shot, RAG, fine-tuning…
3. **Vue d'ensemble de l'architecture** : le schéma complet et ses zones
4. **Docker : comment le labo tourne sur ton PC**
5. **Le backend (API FastAPI)** : modules, base de données, endpoints
6. **L'interface du labo (lab-ui)** : les six onglets
7. **Les modèles et les fournisseurs** : models.yaml, API payantes, gratuites, locales
8. **Le jeu de test** : format, public/privé, empreinte, canari
9. **Le parcours d'un test, de l'écriture au score**
10. **La notation et les métriques** : note humaine, chrF++, intervalles de confiance
11. **Phase 2 : améliorer un petit modèle** : données, few-shot, RAG, QLoRA
12. **La contamination et comment on l'empêche**
13. **La sécurité**
14. **CI/CD : les vérifications automatiques sur GitHub**
15. **Le site public**
16. **Organisation du dépôt** : à quoi sert chaque dossier
17. **Où en est le projet, et la suite**
18. **Glossaire et commandes utiles**

Comment lire ce document

Les chapitres 1 à 3 donnent la vue d'ensemble : lis-les en premier. Les chapitres suivants détaillent chaque brique et peuvent se lire dans n'importe quel ordre. Les encadrés colorés ont toujours le même rôle :

- **vert** : l'idée clé à retenir ;
- **violet** : une analogie de la vie courante ;
- **bleu** : pourquoi ce choix technique a été fait ;
- **orange** : un point d'attention ;
- **rouge** : une règle non négociable du projet.

1 Le projet en une page

Le problème

Le kabyle (taqbaylit) est parlé par plusieurs millions de personnes, mais il existe très peu de textes kabyles numériques. Or les grands modèles d'IA (ChatGPT, Gemini, Mistral…) apprennent à partir de textes trouvés sur Internet. Résultat : ils maîtrisent mal le kabyle. Ils le confondent avec l'arabe ou d'autres langues berbères, répondent en français, ou inventent des mots.

Personne ne sait mesurer précisément *à quel point* ils le maîtrisent mal, car il n'existe pas de test de référence fiable écrit par une locutrice native. C'est ce que construit ce projet.

L'objectif, en trois phases

Phase	Ce qu'on fait	Résultat attendu
1. Mesurer	Tu écris des tests (questions + bonnes réponses en kabyle). Le labo pose ces questions à plusieurs modèles d'IA. Tu notes leurs réponses à l'aveugle.	Un classement : quel modèle parle le mieux kabyle, dans quelles catégories.
2. Améliorer	On prend un petit modèle gratuit (Qwen3.5 4B) et on essaie trois techniques pour l'améliorer : few-shot, RAG, fine-tuning.	Trois versions améliorées du même modèle.
3. Prouver	On repasse exactement le même test au modèle de départ et à ses versions améliorées, et on compare.	Un gain chiffré et honnête, publié sur le site.

Les quatre règles du projet

Règles non négociables

1. **Tout le kabyle de référence vient de toi.** Aucun texte kabyle du jeu de test n'est écrit, complété ou corrigé par une IA.
2. **Aucun chiffre inventé.** Chaque score vient de vraies réponses de modèles et de tes vraies notes. Tant qu'il n'y en a pas, le site affiche « résultats à venir ».
3. **Zéro contamination.** Les tests ne servent jamais à entraîner ou aider un modèle (chapitre 12).
4. **Aucun secret sur GitHub.** Les clés d'API restent dans le fichier `.env`, sur ton PC (chapitre 13).

Les deux grandes parties du logiciel

Le labo (privé, sur ton PC)

Une application web qui tourne uniquement sur ta machine. Tu y écris les tests, lances les modèles, notes les réponses et vois les résultats. Il n'est jamais mis en ligne, car il utilise tes clés d'API et contient tes tests privés.

Le site public (sur Internet)

Un site vitrine en français et en anglais, hébergé gratuitement sur GitHub Pages. Il affiche seulement les résultats que tu choisis d'exporter. C'est la partie « portfolio ».

2 Les notions d'IA à connaître

Ce chapitre explique les mots qui reviennent partout dans le projet. Pas besoin de tout retenir d'un coup : reviens-y quand un mot bloque.

LLM (grand modèle de langage)

Un LLM est un programme qui a lu une quantité gigantesque de textes et qui a appris une seule chose : **deviner le mot suivant**. En répétant cette devinette mot après mot, il écrit des phrases, traduit, répond à des questions. ChatGPT (OpenAI), Gemini (Google), Mistral, Qwen, Gemma sont des LLM.

> **Analogie**
>
> Imagine quelqu'un qui aurait lu des millions de livres sans jamais parler. Il sait très bien comment les phrases « continuent » d'habitude. S'il a lu beaucoup de français, il écrit un bon français. S'il n'a presque jamais vu de kabyle, il improvise, et c'est là que viennent les mots inventés.

Les paramètres. La « mémoire » d'un modèle est faite de nombres appelés paramètres. On parle de « 4B » pour 4 milliards (billions en anglais). Plus il y en a, plus le modèle est capable, mais plus il demande de mémoire et de puissance. Les modèles commerciaux en ont des centaines de milliards. Qwen3.5 4B en a environ 4,7 milliards : c'est un « petit » modèle qui tient sur un PC.

Token

Un modèle ne lit pas des lettres ni des mots, mais des **tokens** : des morceaux de mots. « maison » peut être un seul token, un mot rare peut en faire quatre. Les fournisseurs facturent au token (prix par million de tokens). Le kabyle, peu présent à l'entraînement, est découpé en beaucoup de petits tokens : il coûte plus cher et le modèle le « voit » moins bien. C'est pour cela que le labo estime les coûts de façon prudente (2,5 caractères par token).

Prompt, message système, température

- **Prompt** : le texte qu'on envoie au modèle, par exemple la consigne d'un test.
- **Message système** : une instruction cachée placée avant la conversation (« tu es un assistant qui… »). Le benchmark n'en met pas, pour que tous les modèles reçoivent exactement la même chose. Seule la variante RAG en utilise un (chapitre 11).
- **Température** : le degré de hasard dans le choix des mots. À 0, le modèle choisit toujours le mot le plus probable : ses réponses sont stables et reproductibles. Le projet met la température à 0 partout où c'est possible.

API et fournisseur

Une **API** est une porte d'entrée pour qu'un programme parle à un autre programme. Les entreprises qui hébergent les modèles (les **fournisseurs** : OpenAI, Google, Mistral, OpenRouter) offrent une API : le labo envoie une question par Internet, le fournisseur renvoie la réponse. Pour s'identifier, il faut une **clé d'API**, une sorte de mot de passe personnel. Elle doit rester secrète, car quelqu'un qui la vole peut dépenser ton argent.

Modèle local, Ollama, inférence, GPU

Certains modèles sont **open source** : on peut les télécharger et les faire tourner soi-même, gratuitement. **Ollama** est le logiciel qui fait tourner ces modèles sur ton PC et leur donne une API, comme un fournisseur, mais local. Faire répondre un modèle s'appelle l'**inférence**.

Les calculs d'un modèle sont des multiplications de grands tableaux de nombres. Une carte graphique (**GPU**, surtout NVIDIA) les fait des dizaines de fois plus vite qu'un processeur (**CPU**). Ton PC n'a pas de GPU NVIDIA et a 8 Go de RAM : un modèle local y tourne, mais lentement, et la mémoire est juste. C'est pourquoi les étapes lourdes sont prévues sur Google Colab, qui prête un GPU gratuitement.

Quantification : pour qu'un modèle prenne moins de mémoire, on arrondit ses paramètres (par exemple sur 4 bits au lieu de 16). Il perd un peu en qualité mais devient 3 à 4 fois plus léger. Le format de fichier **GGUF** stocke ces modèles quantifiés pour Ollama.

Benchmark

Un **benchmark** est un examen standardisé pour les modèles : les mêmes questions, posées de la même façon, notées avec les mêmes règles. Il permet de comparer des modèles entre eux, ou un modèle avant et après amélioration. Ce projet est un benchmark du kabyle.

Les trois façons d'améliorer un modèle

Technique	Principe	Analogie
Few-shot	On met quelques exemples de traductions déjà faites dans la question, juste avant la vraie demande. Le modèle ne change pas, il s'inspire des exemples.	Montrer trois exercices corrigés à un élève juste avant l'examen.
RAG (génération augmentée par recherche)	On cherche automatiquement, dans une grande bibliothèque de phrases, celles qui ressemblent le plus à la question, et on les fournit au modèle comme aide-mémoire.	Un examen à livre ouvert : l'élève a droit à un dictionnaire bien rangé.
Fine-tuning (QLoRA)	On continue l'entraînement du modèle sur des milliers de phrases kabyles. Cette fois, le modèle lui-même change.	Envoyer l'élève suivre des cours de kabyle pendant plusieurs semaines.

Embedding (plongement)

Un **embedding** transforme une phrase en une liste de nombres (ici 1 024 nombres) qui résume son *sens*. Deux phrases de sens proche ont des listes de nombres proches, même si elles n'ont aucun mot en commun. C'est ce qui permet au RAG de retrouver « les phrases qui ressemblent à la question ». Le projet utilise le modèle d'embedding **bge-m3**, multilingue, servi par Ollama.

> **Analogie**
>
> C'est comme donner à chaque phrase une adresse GPS dans une « carte des sens ». « J'ai faim » et « je voudrais manger » habitent dans le même quartier. Chercher les phrases proches revient à chercher les voisins sur la carte.

LoRA et QLoRA

Réentraîner les 4,7 milliards de paramètres d'un modèle demanderait une machine énorme. **LoRA** ne modifie pas le modèle d'origine : il ajoute à côté une petite « rustine » de quelques millions de paramètres, et n'entraîne que cette rustine. **QLoRA** fait la même chose sur le modèle quantifié en 4 bits, ce qui fait tenir l'entraînement sur un GPU gratuit de Colab.

Contamination

Si un modèle a vu les questions d'un examen pendant son entraînement, sa bonne note ne prouve rien : il récite. C'est la **contamination**. Dans ce projet, c'est le risque n°1 de la phase 2 : il ne faut surtout pas que tes tests se retrouvent dans les exemples du few-shot, la bibliothèque du RAG ou les données de fine-tuning. Le chapitre 12 explique comment c'est empêché.

3 Vue d'ensemble de l'architecture

Le schéma ci-dessous montre toutes les briques et comment les informations circulent. Les chapitres suivants les détaillent une par une.

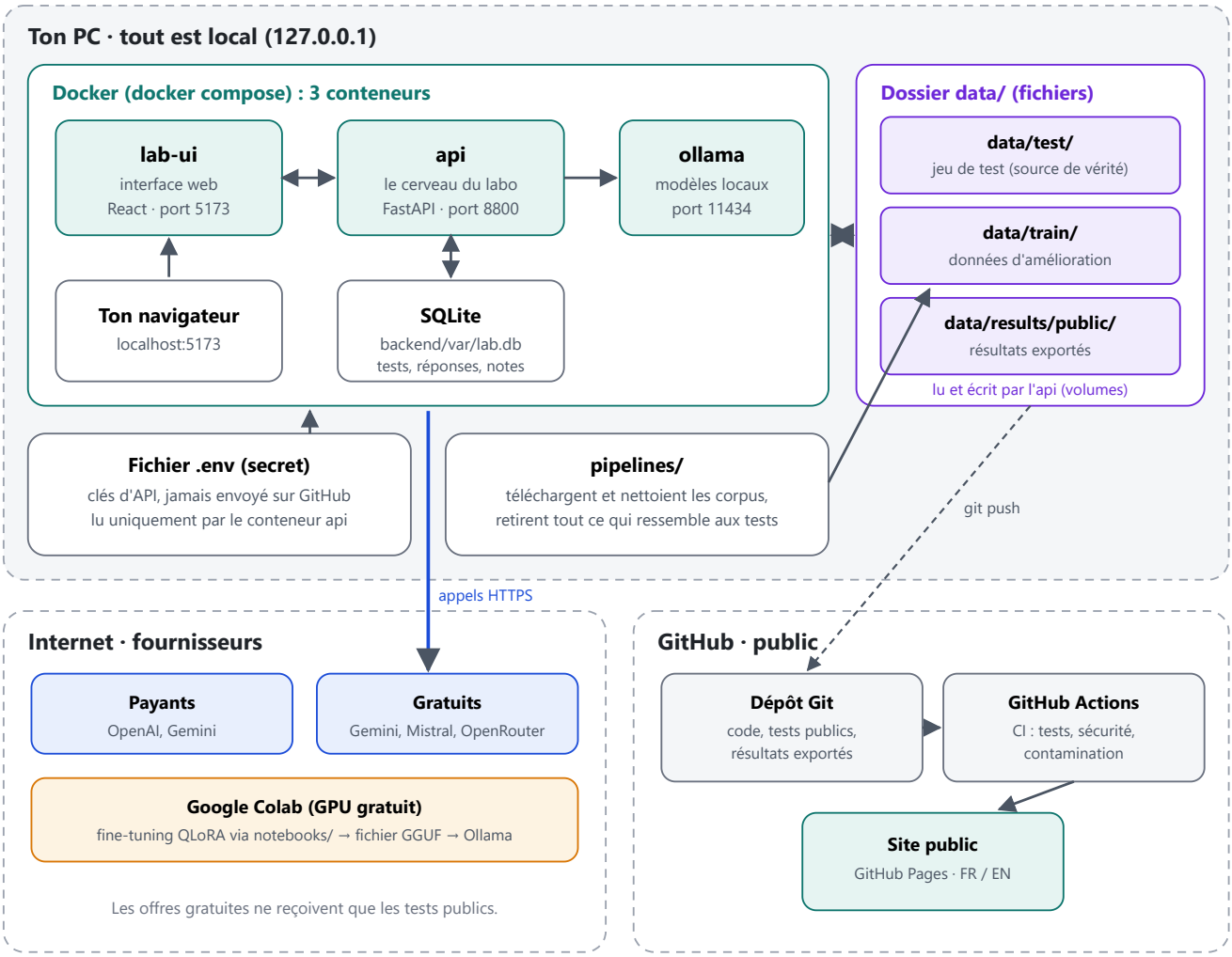


Figure 1 · Architecture générale. Flèches pleines : appels entre programmes. Pointillés : lecture ou écriture de fichiers.

Les trois zones à retenir

Zone	Contenu	Qui peut y accéder
Ton PC	Le labo (3 conteneurs Docker), la base SQLite, les tests privés, le fichier `.env` avec les clés.	Toi seule. Le labo n'écoute que sur 127.0.0.1, l'adresse « ma propre machine ».
Les fournisseurs	Les modèles commerciaux et gratuits, interrogés par Internet. Colab pour le fine-tuning.	Ils ne voient que les questions qu'on leur envoie.
GitHub	Le code, les tests publics, les résultats exportés, le site.	Tout le monde (dépôt public).

L'idée clé

Le labo est un outil de travail privé ; le site est une vitrine publique. Entre les deux, une seule porte contrôlée : l'export des résultats, qui ne laisse passer qu'une liste précise de champs (chapitre 13).

4 Docker : comment le labo tourne sur ton PC

Pourquoi Docker

Le labo a besoin de Python (pour l'API) et de Node.js (pour l'interface). Ton PC n'a ni l'un ni l'autre, et Windows y bloque certains programmes non signés (Smart App Control). **Docker** résout ces deux problèmes : chaque partie tourne dans un **conteneur**, une petite machine Linux isolée qui contient déjà tout ce qu'il faut.

> **Analogie**
>
> Un conteneur, c'est comme une boîte-repas préparée : tout le nécessaire est dedans, et elle se réchauffe de la même façon dans n'importe quelle cuisine. Le fichier `docker-compose.yml` est le menu qui dit quelles boîtes ouvrir et comment les relier.

Les trois services de `docker-compose.yml`

Service	Rôle	Adresse	Démarrage
`api`	Le backend Python : il stocke les tests, appelle les modèles, calcule les scores. Image construite depuis `backend/Dockerfile`.	127.0.0.1:8800 doc interactive : /docs	toujours
`lab-ui`	L'interface web. Image `node:22-alpine` ; installe les dépendances puis lance Vite en mode développement.	localhost:5173	toujours
`ollama`	Fait tourner les modèles locaux (Qwen, bge-m3…). Les modèles téléchargés sont gardés dans le volume `ollama-models`.	127.0.0.1:11434	seulement avec `--profile ollama`

Les volumes : le lien entre le conteneur et tes fichiers

Un conteneur est jetable : si on le supprime, ce qu'il contient disparaît. Pour garder les données, on « monte » des dossiers de ton PC dans le conteneur :

- `backend/var` → la base SQLite `lab.db` (tes réponses et tes notes) ;
- `data/` → le jeu de test, les données d'amélioration, les résultats ;
- `backend/app`, `pipelines/`, `scripts/` … → le code, monté « à chaud » : une modification du code est prise en compte immédiatement (option `--reload`).

Deux volumes sont gérés par Docker lui-même : `lab-ui-node-modules` (dépendances JavaScript, plus rapide sous Windows) et `ollama-models` (les modèles téléchargés, plusieurs Go).

Pourquoi les ports commencent par 127.0.0.1

`127.0.0.1:8800:8000` veut dire : « le port 8000 du conteneur est accessible sur le port 8800, mais seulement depuis cette machine ». Sans `127.0.0.1`, n'importe quel appareil de ton Wi-Fi pourrait utiliser ton labo, et donc tes clés d'API.

Les commandes de base

```
docker compose up -d                      # démarre api + lab-ui
docker compose --profile ollama up -d     # démarre aussi Ollama
docker compose down                       # arrête tout (les données restent)
docker compose logs -f api                # affiche le journal de l'API
docker compose restart api                # après une modification de .env
```

5 Le backend (API FastAPI)

Le backend est écrit en **Python** avec **FastAPI**, un cadre pour créer des API web. L'interface ne fait aucun calcul : elle demande tout à l'API par des requêtes HTTP, et l'API répond en JSON (un format texte structuré).

Les modules de `backend/app/`

Fichier	Rôle
`main.py`	Point de départ : crée l'application, applique les migrations de la base, branche les routes et les protections.
`config.py`	Lit la configuration (fichier `.env` et variables d'environnement) : chemins, clés, plafond de coût. Une clé vide compte comme absente.
`models.py`	Définit un test (`BenchItem`) : catégorie, consigne, références, public/privé, statut.
`run_models.py`	Définit une exécution (`Run`), ses tests figés (`RunItem`), chaque réponse (`ModelResponse`), chaque note (`Grade`) et les exemples marquants (`Favorite`).
`testset.py`	Import/export du jeu de test, écriture du JSONL dans le dépôt, calcul de l'empreinte SHA-256 et du manifeste.
`registry.py`	Charge la liste des modèles depuis `config/models.yaml` et la vérifie.
`providers.py`	Parle aux fournisseurs : une seule classe pour tous ceux qui partagent le format OpenAI (OpenAI, Mistral, OpenRouter, Gemini), une pour Ollama, et une factice pour les tests automatiques.
`variants.py`	Construit le message envoyé au modèle selon la variante : simple, few-shot ou RAG.
`runner.py`	Le moteur d'exécution : sélectionne les tests, appelle les modèles en parallèle, relance en cas d'erreur, applique le plafond de coût.
`scoring.py`	Scores automatiques : chrF++, BLEU, écriture et langue de la réponse.
`contamination.py`	Détecte si un texte ressemble à un test (chapitre 12).
`results.py`	Agrège les notes : moyennes, intervalles de confiance, gains avant/après, étiquettes d'erreur.
`export_public.py`	Écrit `results.json` pour le site, avec une liste blanche de champs.
`setup.py`	Construit la liste de préparation (onglet Préparation).
`security.py`	Protections de l'API locale (chapitre 13).
`api/*.py`	Les routes HTTP, regroupées par thème : tests, mode guidé, jeu de test, exécutions, notation, résultats.

La base de données SQLite

SQLite est une base de données contenue dans un seul fichier (`backend/var/lab.db`), sans serveur à installer. Elle contient cinq tables principales :

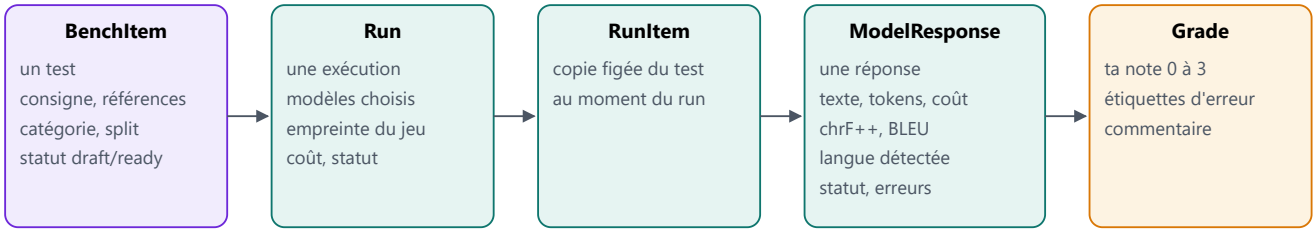


Figure 2 · Les tables de la base et leurs liens.

Le schéma de la base évolue avec **Alembic** (des « migrations » numérotées), appliqué automatiquement au démarrage. Un test automatique vérifie que les migrations correspondent toujours au code.

> **Attention : `lab.db` n'est pas dans Git**
>
> Les réponses des modèles et tes notes ne peuvent pas être recréées (relancer un modèle coûte de l'argent et donne parfois une réponse différente). Sauvegarde régulièrement avec `docker compose run --rm api python -m scripts.backup` , puis copie `backend/var/backups/` sur une clé USB ou un stockage externe.

Le moteur d'exécution (runner)

Quand tu lances une exécution avec 3 modèles et 52 tests, le runner :

1. **crée d'avance** 3 × 52 = 156 lignes « en attente » dans la base, une par couple (test, modèle) ;
2. envoie les questions **en parallèle**, au maximum 2 à la fois par fournisseur (`MAX_CONCURRENCY_PER_PROVIDER`) ;
3. en cas d'erreur temporaire (trop de requêtes, serveur en panne, réseau), **attend puis réessaie**, avec 4 tentatives au maximum (`MAX_ATTEMPTS`) et une attente qui double à chaque fois ;
4. **s'arrête** si le coût estimé dépasse le plafond (`MAX_RUN_COST_USD` , 5 $ par défaut) ;
5. calcule les scores automatiques dès que chaque réponse arrive.

> **Pourquoi des lignes créées d'avance**
>
> Si le PC s'éteint au milieu d'une exécution, on sait exactement ce qui manque : il suffit de « reprendre », et seules les lignes encore en attente ou en erreur sont rejouées. On ne paie jamais deux fois la même réponse. C'est ce qu'on appelle l'**idempotence**.

L'interface suit la progression **en direct** grâce aux *Server-Sent Events* : l'API envoie une mise à jour par seconde sans que l'interface ait besoin de redemander.

6 L'interface du labo (lab-ui)

L'interface est écrite en **React** avec **TypeScript** (du JavaScript avec vérification des types), assemblée par **Vite** et stylée avec **Tailwind CSS**. Elle a six onglets, un par étape du travail :

Onglet	À quoi il sert	Fichier
Préparation	La page d'accueil : ce qui est prêt et ce qui manque (tests, clés présentes ou non, Ollama, données, index RAG), avec la commande à lancer pour chaque point. La valeur des clés n'est jamais affichée.	`pages/SetupPage.tsx`
Mode guidé	Propose les consignes en français une par une (fichier `data/prompts/guided_fr.yaml`) ; tu saisis la réponse kabyle. Ctrl+Entrée enregistre.	`pages/GuidedPage.tsx`
Tests	Liste, filtres, modification, import/export (JSONL, JSON, CSV), et « Enregistrer dans le dépôt ».	`pages/TestsPage.tsx`
Exécutions	Choisir les modèles, les catégories, public/privé, le mode test ; voir l'estimation du coût ; lancer et suivre en direct ; reprendre.	`pages/RunsPage.tsx`
Notation	Les réponses arrivent mélangées et anonymes. Touches 0 à 3 pour noter, lettres pour les étiquettes d'erreur, « Révéler » pour voir le modèle après coup.	`pages/GradePage.tsx`
Résultats	Classement, graphiques par catégorie, gains avant/après, erreurs fréquentes, exploration des réponses, export vers le site.	`pages/ResultsPage.tsx`

La **barre de caractères** (`components/CharBar.tsx`) insère d'un clic les lettres kabyles absentes du clavier (ɛ, ɣ, ḥ, ṭ, ẓ, č, ǧ…) et les lettres tifinagh. La liste se modifie dans `lib/charsets.ts` .

> **Comment l'interface parle à l'API**
>
> L'interface appelle `/api/...` sur sa propre adresse (localhost:5173). Vite transmet ces appels au conteneur `api` : c'est un « proxy ». Le navigateur ne parle donc qu'à une seule adresse, ce qui simplifie la sécurité.

7 Les modèles et les fournisseurs

Le fichier `backend/config/models.yaml`

Tous les modèles du labo sont déclarés dans ce fichier. Ajouter un modèle, c'est ajouter quelques lignes ; aucun code à écrire tant que le fournisseur est connu. Exemple :

```
- key: gemini-flash-lite              # identifiant interne, unique
  label: Gemini Flash-Lite            # nom affiché
  provider: gemini                    # quel fournisseur appeler
  model: gemini-flash-lite-latest     # nom exact chez le fournisseur
  params: { max_tokens: 2048, temperature: 0 }   # réglages envoyés tels quels
  pricing: { input: 0, output: 0 }    # prix en $ par million de tokens (null si inconnu)
  public_only: true                   # offre gratuite : tests publics seulement
```

Une **variante** (few-shot, RAG, fine-tunée) est un modèle comme un autre, avec deux champs en plus : `base` (son modèle de départ) et `variant`. C'est ce lien qui permet de calculer automatiquement le gain.

Les modèles configurés aujourd'hui

Fournisseur	Modèles	Coût	Clé (.env)
OpenAI	GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna	payant	`OPENAI_API_KEY`
Mistral	Mistral Large, Mistral Small	plan gratuit « Experiment »	`MISTRAL_API_KEY`
Google Gemini	Gemini Flash, Gemini Flash-Lite	plan gratuit	`GEMINI_API_KEY`
OpenRouter	Gemma 4 31B, Nemotron 3 Ultra 550B, Inkling (versions `:free`)	gratuit, quotas	`OPENROUTER_API_KEY`
Ollama (local)	Qwen3.5 4B, Gemma 4 E4B, Apertus 8B + les 3 variantes de Qwen	gratuit (ton PC)	aucune

Un modèle dont la clé manque apparaît grisé : il est simplement ignoré, rien ne plante.

Une seule classe pour quatre fournisseurs

OpenAI, Mistral, OpenRouter et Gemini acceptent tous le même format de requête (« chat/completions ») ; seule l'adresse change. Le fichier `providers.py` a donc une seule classe `OpenAICompatibleProvider` pour les quatre. Ollama a son propre format.

> **Pourquoi les relances sont gérées par le runner et pas par les SDK**
>
> Chaque SDK a sa propre politique de relance. En les désactivant et en relançant nous-mêmes, tous les fournisseurs sont traités exactement de la même façon : c'est plus juste pour un benchmark.

Le verrou `public_only` (offres gratuites)

Les offres gratuites peuvent se réserver le droit d'utiliser les questions reçues pour entraîner leurs futurs modèles. Pour les tests publics, ce n'est pas grave : ils sont déjà sur GitHub. Pour les tests privés, ce serait une fuite. Donc :

- dans `runner.py`, l'API **refuse** toute exécution qui enverrait des tests privés à un modèle marqué `public_only` ;
- dans l'interface, sélectionner un tel modèle **bloque** le choix sur « Publique » et affiche le badge « gratuit · tests publics » ;
- un test automatique vérifie qu'aucune consigne privée n'est envoyée.

Pas de repli automatique

Si un modèle refuse de répondre, le refus est enregistré tel quel. Le labo ne demande jamais à un autre modèle de répondre à sa place : dans un benchmark, une réponse doit toujours venir du modèle évalué.

8 Le jeu de test

Ce qu'il contient aujourd'hui

Catégorie	Ce qui est mesuré	Tests
Traduction français → kabyle	Le modèle sait-il écrire en kabyle ?	32
Traduction kabyle → français	Comprend-il le kabyle écrit ?	5
Compréhension	Un court texte kabyle, puis une question	4
Culture	Proverbes, expressions, coutumes	4
Écriture	Alphabet latin kabyle et tifinagh	3
Conversation	Répond-il vraiment en kabyle ?	4
Total	**41 publics (sur GitHub) + 11 privés (21 %, sur ton PC uniquement)**	**52**

Un test, concrètement

Chaque test est une ligne de texte au format **JSON**. Le fichier `data/test/kab-bench.v1.jsonl` en contient un par ligne (format JSONL). Les champs principaux :

```
{ "id": "tr-fr-kab-0001",                identifiant unique
  "category": "translation_fr_kab",      catégorie
  "prompt": "Traduis en kabyle : « … »", consigne envoyée telle quelle au modèle
  "source_text": "…",                    texte à traduire
  "references": ["…", "…"],              tes bonnes réponses (une ou plusieurs)
  "grading": "auto+human",               notation automatique et humaine
  "expected_lang": "kab",                langue attendue de la réponse
  "split": "public",                     public ou private
  "status": "ready" }                    seuls les tests « ready » sont évalués
```

Un test ne peut passer en `ready` que s'il est complet : une référence s'il est noté automatiquement, un texte source pour une traduction, un texte de contexte pour une compréhension.

Public et privé

Environ 20 % des tests sont gardés **privés**, dans `data/test/private/`, un dossier que Git ignore. Pourquoi ? Les tests publics finiront sur Internet, et un jour peut-être dans l'entraînement d'un futur modèle. Les tests privés, eux, restent inédits : ils permettent toujours de vérifier qu'un modèle ne fait pas que réciter.

La source de vérité : les fichiers, pas la base

Tu saisis les tests dans le labo (donc dans SQLite), puis « Enregistrer dans le dépôt » les écrit dans les fichiers JSONL. Ce sont ces fichiers, versionnés dans Git, qui font foi. La base peut être supprimée et rechargée à tout moment depuis eux.

L'empreinte et la chaîne canari

Le fichier `data/test/MANIFEST.json` contient :

- la **version** du jeu (actuellement 1.0.0) ;
- une **empreinte SHA-256** : un code de 64 caractères calculé à partir du contenu des tests. Si une seule référence change, l'empreinte change. Chaque exécution enregistre cette empreinte : deux scores ne sont comparés que s'ils ont la même, c'est-à-dire s'ils portent exactement sur le même examen ;
- une **chaîne canari** : un code unique sans signification. Si un jour un modèle est capable de le réciter, c'est la preuve qu'il a été entraîné sur ce jeu de test.

> **Analogie**
>
> L'empreinte est comme le numéro de version d'un sujet d'examen : on ne compare pas les notes de deux classes qui n'ont pas eu le même sujet. Le canari est comme une fausse entrée qu'un cartographe ajoute à sa carte pour repérer qui la copie.

Le texte est aussi **normalisé en Unicode NFC** : une lettre comme « ḥ » peut s'écrire de deux façons invisibles à l'œil ; la normalisation les rend identiques, pour que la comparaison avec la réponse du modèle soit juste.

9 Le parcours d'un test, de l'écriture au score

Voici ce qui se passe, étape par étape, quand un test traverse tout le système.

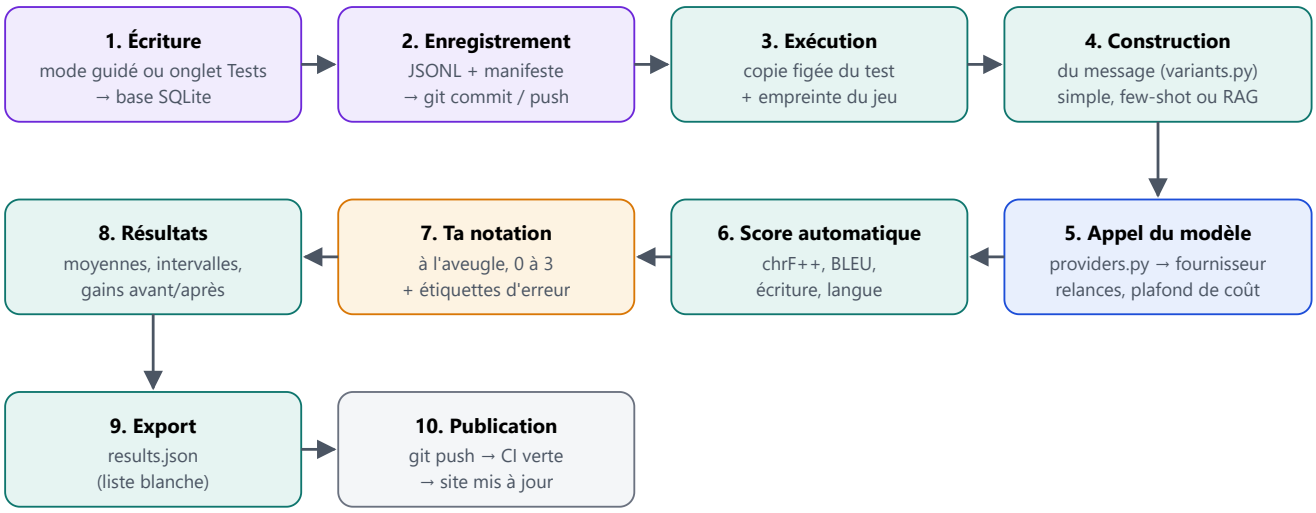


Figure 3 · Le parcours complet d'un test.

Les étapes en détail

1. **Écriture.** Tu saisis la consigne et ta référence kabyle. Le test est d'abord en `draft`, puis `ready` quand il est complet.
2. **Enregistrement.** « Enregistrer dans le dépôt » écrit le JSONL public, le JSONL privé (ignoré par Git) et le manifeste. Tu commits et pushes la partie publique.
3. **Exécution.** Le run garde une **copie figée** de chaque test. Si tu corriges un test plus tard, les anciennes réponses restent interprétées avec la version qu'elles ont reçue.
4. **Construction du message.** Pour un modèle simple, c'est la consigne telle quelle (précédée du texte de compréhension s'il y en a un). Pour une variante, `variants.py` ajoute des exemples (few-shot) ou un aide-mémoire (RAG).
5. **Appel du modèle.** Le runner envoie la requête, relance si besoin, et enregistre la réponse brute, les tokens, le coût, la durée et les paramètres réels.
6. **Score automatique.** chrF++ et BLEU comparent la réponse à tes références ; on détecte aussi l'écriture (latin, tifinagh, arabe) et la langue.
7. **Notation humaine.** Tu vois la consigne, tes références et la réponse, sans savoir quel modèle l'a écrite. Tu notes de 0 à 3 et ajoutes des étiquettes d'erreur.
8. **Résultats.** `results.py` calcule les moyennes par modèle et par catégorie, les intervalles de confiance et les gains des variantes.
9. **Export.** Seuls certains champs sont écrits dans `data/results/public/results.json`. Les tests privés ne comptent que dans les moyennes ; leur texte n'est jamais exporté.
10. **Publication.** Après un `git push`, si toutes les vérifications passent, le site se reconstruit automatiquement.

10 La notation et les métriques

La note humaine : la mesure qui fait foi

Note	Signification
0	Faux, hors sujet, incompréhensible ou dans une autre langue
1	Partiellement correct, erreurs importantes
2	Correct sur le fond, erreurs mineures (orthographe, tournure)
3	Correct et naturel

La note est aussi exprimée en pourcentage : note ÷ 3 × 100. Une moyenne de 1,5 vaut donc 50 %.

> **Pourquoi « à l'aveugle »**
> Si tu savais qu'une réponse vient de Gemini ou de GPT, tu pourrais inconsciemment être plus indulgente ou plus sévère. L'interface ne reçoit donc ni le nom du modèle, ni les scores automatiques, et les réponses de tous les modèles sont mélangées au hasard. C'est le principe des essais cliniques en « double aveugle ».

Les **étiquettes d'erreur** décrivent *pourquoi* une réponse est mauvaise : mauvaise langue, confusion avec l'arabe, autre langue berbère, mots inventés, orthographe, grammaire, contresens, trop littéral, incomplet, mauvaise écriture, refus. Elles permettent de dire non seulement « ce modèle a 40 % », mais aussi « il échoue surtout parce qu'il invente des mots ».

chrF++ : la mesure automatique principale

chrF++ compare la réponse du modèle à ta référence en regardant les **morceaux de caractères** qu'elles ont en commun (des séquences de 1 à 6 lettres), plus les paires de mots. Le score va de 0 (rien en commun) à 100 (identique).

> **Exemple (en français, pour l'illustration)**
> Référence : « les enfants jouent dehors ». Réponse : « les enfant jouent dehors ».
> Une mesure par mots entiers (comme BLEU) voit un mot faux sur quatre. chrF++ voit que presque tous les morceaux de lettres sont là, et donne un score élevé : l'erreur ne porte que sur un « s ».

C'est exactement ce qu'il faut pour le kabyle : la langue colle beaucoup de petits éléments aux mots (préfixes, suffixes) et son orthographe varie. Une petite erreur sur un affixe ne doit pas annuler tout le mot. chrF++ est d'ailleurs la mesure de référence pour les langues peu dotées. S'il y a plusieurs références, on garde la meilleure correspondance.

BLEU est calculé aussi, pour pouvoir comparer avec d'autres travaux, mais il n'est jamais utilisé pour classer les modèles : il est peu fiable sur des phrases courtes.

Pourquoi pas COMET, une mesure plus moderne ? Elle repose sur un modèle (XLM-R) qui connaît mal le kabyle : ses scores seraient trompeurs.

Langue et écriture de la réponse

- **L'écriture** est détectée de façon certaine en comptant les lettres latines, tifinagh ou arabes.
- **La langue** est détectée par GlotLID (un outil qui reconnaît plus de 1 600 langues, dont le kabyle) s'il est installé. Sinon, une règle prudente repère seulement les échecs évidents (écriture arabe, mots français courants). Elle ne déclare jamais qu'une réponse est en kabyle. GlotLID n'est pas installé pour l'instant.

Intervalle de confiance : « à quel point je peux me fier à ce chiffre »

Avec 52 tests, une moyenne de 60 % pourrait être 55 % ou 65 % avec d'autres tests du même genre. L'**intervalle de confiance à 95 %** donne cette fourchette. Le projet le calcule par **bootstrap** :

1. on tire au hasard 52 tests parmi les 52, *avec remise* (certains reviennent deux fois, d'autres pas du tout) ;
2. on recalcule la moyenne ;
3. on recommence 1 000 fois ;
4. on garde la fourchette qui contient 95 % de ces 1 000 moyennes.

> **À retenir**
> Si les fourchettes de deux modèles se chevauchent largement, on ne peut pas dire honnêtement que l'un est meilleur que l'autre. Le site l'affiche pour éviter les conclusions hâtives.

Le gain d'une variante (comparaison appariée)

Pour mesurer si le few-shot améliore Qwen, on ne compare que les tests notés **pour les deux** (Qwen de base et Qwen few-shot). Ainsi, on compare la même question avant et après. On calcule :

- le gain en points : % après − % avant (par exemple 45 % → 60 % = +15 points) ;
- le gain relatif : 15 ÷ 45 = +33 % ;
- un intervalle de confiance bootstrap sur ces différences.

Corrélation de Spearman : chrF++ est-il fiable pour le kabyle ?

Le labo mesure si chrF++ classe les réponses dans le même ordre que toi (coefficient ρ, de −1 à 1). Proche de 1 : chrF++ est un bon indicateur rapide pour le kabyle. Proche de 0 : il ne reflète pas ton jugement. C'est en soi un résultat intéressant à publier.

11 Phase 2 : améliorer un petit modèle

Le modèle choisi

Trois petits modèles open source sont candidats ; on les mesure tous, puis on améliore le plus prometteur. **Qwen3.5 4B** est configuré par défaut.

Modèle	Taille	Pourquoi il est candidat
Qwen3.5 4B	4,7 Md	Licence Apache 2.0, annonce 201 langues, rapide sur un GPU gratuit, bien pris en charge par l'outil de fine-tuning Unsloth
Gemma 4 E4B	≈ 4 Md effectifs	Google, pré-entraîné sur plus de 140 langues
Apertus 8B	8 Md	Annonce 1 811 langues ; a probablement vu du kabyle, mais deux fois plus lourd

Les données d'amélioration

Les pipelines (`pipelines/`) préparent les données à partir de corpus publics dont la licence a été vérifiée :

Source	Contenu	Licence
Tatoeba	93 908 paires de phrases français–kabyle traduites par des bénévoles	CC BY 2.0 FR
Common Voice	≈ 180 000 phrases kabyles (sans traduction)	CC0

Résultat du dernier passage (fichier `data/train/processed/STATS.json` , 29 octobre 2026) :

Étape	Nombre
Paires en entrée	93 908
Retirées : doublons / invalides / **proches d'un test**	196 / 67 / **6**
Paires conservées	93 639
Exemples d'entraînement générés (les deux sens de traduction)	187 192, dont 86 retirés car trop proches d'une consigne de test
Phrases monolingues conservées	178 092

```
docker compose run --rm api python -m pipelines.download            # télécharge
docker compose run --rm api python -m pipelines.prepare             # nettoie et décontamine
docker compose run --rm api python -m pipelines.check_contamination  # doit afficher « OK »
```

Technique 1 : few-shot

Avant ta question, on ajoute **5 exemples** de traduction tirés de Tatoeba, présentés comme une conversation déjà faite (question, réponse, question, réponse…). Les exemples choisis sont ceux qui partagent le plus de morceaux de lettres avec la phrase à traduire. Le modèle n'est pas modifié. Variante : `qwen3.5-4b-fewshot` .

Technique 2 : RAG

1. **Une fois** : `pipelines/build_rag_index.py` calcule l'embedding (bge-m3) de chaque paire et enregistre le tout dans `data/train/processed/rag_index/` (un tableau de nombres `embeddings.npy` et le texte `entries.jsonl`).
2. **À chaque question** : on calcule l'embedding de la question, on trouve les **8 paires** les plus proches par un simple calcul (produit scalaire, avec numpy), et on les place dans un message système « Voici des paires de phrases kabyle–français… ». Variante : `qwen3.5-4b-rag`.

> **Pourquoi pas une « base de données vectorielle »**
>
> Pour moins de 100 000 phrases, une multiplication de tableaux prend quelques millisecondes. Une base vectorielle ajouterait un serveur à gérer sans rien apporter. Le fonctionnement reste aussi entièrement transparent.

> **État actuel**
>
> L'index RAG n'est pas encore construit : la construction sur 20 000 paires a été arrêtée faute de mémoire sur ton PC. Il faudra la relancer avec moins de paires, ou sur Colab.

Technique 3 : fine-tuning QLoRA

Le notebook `notebooks/01_qlora_finetune.ipynb` s'ouvre dans Google Colab (GPU gratuit). Il entraîne une rustine LoRA (r = 16) sur Qwen3.5 4B quantifié en 4 bits, avec les exemples décontaminés `sft_train.jsonl`. Puis il exporte le modèle au format GGUF, que tu importes dans Ollama sous le nom `kab-qwen3.5-4b`. Variante : `qwen3.5-4b-lora`.

Estimation (non mesurée) : 1 à 1,5 h pour 60 000 exemples sur le GPU gratuit de Colab ou Kaggle.

Mesurer le gain (phase 3)

Une seule exécution avec Qwen de base et ses trois variantes, sur tous les tests. Tu notes les réponses mélangées à l'aveugle, et l'onglet Résultats calcule les gains appariés par catégorie.

12 La contamination et comment on l'empêche

Règle

Aucune phrase de tes tests (publics ou privés) ne doit se retrouver, même presque à l'identique, dans les données du few-shot, du RAG ou du fine-tuning.

Comment on reconnaît une phrase « trop proche »

Le fichier `backend/app/contamination.py` fait deux contrôles :

1. **Doublon exact**, après normalisation : on met tout en minuscules, on retire la ponctuation et les espaces en trop, on applique NFC. « Bonjour, Ali ! » et « bonjour ali » deviennent identiques.
2. **Quasi-doublon** : on découpe chaque phrase en morceaux de 5 caractères qui se chevauchent (« bonjo », « onjou », « njour »…), puis on calcule la **similarité de Jaccard** = morceaux communs ÷ morceaux au total. Si elle atteint **0,8** (80 % en commun), la phrase est considérée comme contaminée. Ce contrôle ne s'applique qu'aux textes d'au moins 15 caractères ; en dessous, seul le doublon exact a du sens.

Sont protégés : la consigne, le texte source, le texte de contexte et toutes les références de chaque test.

Les trois niveaux de protection

Niveau	Ce qui se passe
1. Pipeline	`prepare.py` retire des données toute phrase ou consigne générée trop proche d'un test, et enregistre l'**empreinte** du jeu de test qu'il a utilisé.
2. Labo	Avant de lancer une variante few-shot ou RAG, le labo revérifie toutes ses données. Au moindre recoupement, la variante **refuse de démarrer**. L'onglet Préparation signale aussi si les données ont été préparées avec une ancienne version du jeu de test.
3. CI (GitHub)	À chaque push, GitHub rejoue la préparation et le contrôle : si un recoupement apparaît, la CI échoue.

Exemple réel

C'est le niveau 3 qui a détecté que 86 consignes d'entraînement générées ressemblaient trop à des consignes de test (mêmes gabarits, phrases courtes). La correction a ajouté un filtre dans `prepare.py` (décision n° 31 du journal).

À faire après chaque ajout de tests

Relancer `pipelines.prepare` (et `pipelines.build_rag_index`), sinon le labo bloque les variantes few-shot et RAG.

13 La sécurité

Les clés d'API

- Elles vivent uniquement dans `.env`, à la racine du projet. Le fichier `.gitignore` empêche Git de l'envoyer.
- `.env.example` (lui, sur GitHub) montre les noms des variables, toujours vides.
- L'API ne renvoie jamais la valeur d'une clé ; elle dit seulement « présente » ou « absente ». Un test automatique le vérifie.

Défense en profondeur contre les fuites de secrets

Barrière	Rôle
`.gitignore`	Git ignore `.env`, la base, les tests privés, les données téléchargées.
Hook pre-commit	Avant chaque commit sur ton PC : **gitleaks** cherche des secrets dans les fichiers modifiés et bloque tout `.env`.
CI (job « secrets »)	gitleaks analyse tout l'historique ; la CI échoue si un `.env` ou un test privé est suivi par Git.

Si une barrière rate quelque chose, la suivante le rattrape.

Protéger le labo local

Même un outil local peut être attaqué *à travers ton navigateur*. Une page web malveillante ouverte dans un autre onglet pourrait essayer d'envoyer une requête à `localhost:8800` pour lancer une exécution payante. Deux protections dans `security.py` :

- **Refus des écritures venant d'un autre site** : toute requête qui modifie quelque chose (POST, PUT, DELETE) est refusée si elle ne vient pas de l'interface du labo (en-têtes `Origin` et `Sec-Fetch-Site`). Cela bloque les attaques *CSRF*.
- **Liste d'hôtes autorisés** : l'API n'accepte que les noms `localhost`, `127.0.0.1` et `api`. Cela bloque une attaque appelée *DNS rebinding*, où un faux nom de domaine pointe vers ta machine.

Ce qui peut sortir de ton PC, et ce qui ne le peut pas

Donnée	Où elle peut aller
Tests publics	GitHub, site, tous les fournisseurs (y compris gratuits)
Tests privés	Seulement Ollama (local) et les fournisseurs payants. Jamais GitHub, jamais le site, jamais une offre gratuite (verrou `public_only`)
Clés d'API	Nulle part, sauf vers leur propre fournisseur
Résultats	Le site, uniquement via l'export à liste blanche

Les dépendances

Le projet utilise des bibliothèques écrites par d'autres. Une faille découverte dans l'une d'elles est une faille du projet. Trois outils surveillent cela : **Dependabot** (GitHub propose automatiquement les mises à jour), **npm audit** et **pip-audit** (la CI échoue sur toute faille grave), et **CodeQL** (analyse le code à la recherche d'erreurs de sécurité). Lors de l'audit du 29 octobre, les 21 alertes Dependabot ont été corrigées ; il reste 0 alerte.

Le fichier `SECURITY.md` explique comment signaler une faille en privé.

14 CI/CD : les vérifications automatiques sur GitHub

CI (intégration continue) : à chaque `git push`, GitHub lance automatiquement une série de vérifications sur une machine à lui. **CD** (déploiement continu) : si tout est vert sur la branche `main`, le site est mis à jour automatiquement. Ces « workflows » sont décrits dans `.github/workflows/`.

`ci.yml` : à chaque push, sur toutes les branches

Job	Ce qu'il vérifie
secrets	Aucun secret dans l'historique (gitleaks), aucun `.env` ni test privé suivi par Git
backend	Style du code Python (ruff) et les tests automatiques (pytest, environ 75 tests)
contamination	Rejoue la préparation des données et le contrôle de contamination
lab-ui	Lint, vérification des types, tests et compilation de l'interface
site	Compilation du site public
audit	Failles connues dans les dépendances (npm audit, pip-audit)
dependency-review	Sur les pull requests : nouvelles dépendances vulnérables
docker	L'image de l'API se construit correctement

`codeql.yml` et `pages.yml`

- **CodeQL** analyse le code Python, TypeScript et les workflows, à chaque push et chaque semaine.
- **Pages** se déclenche seulement quand la CI de `main` a réussi, et déploie exactement le commit qui a été vérifié. Un état cassé ne peut donc jamais être publié.

> **Pourquoi c'est important pour un portfolio**
>
> Le badge « CI passing » sur le README montre à un recruteur que le projet est testé automatiquement, sécurisé et déployé proprement : ce sont des pratiques DevOps professionnelles.

15 Le site public

Le site (`site/`) est construit avec **Astro**, qui génère des pages HTML statiques : pas de serveur, pas de base de données, donc rien à pirater et un affichage très rapide. Les graphiques interactifs sont des petits composants React (`Charts.tsx` , bibliothèque Recharts) ; c'est ce qu'Astro appelle des « îlots ».

- **Bilingue** : `pages/index.astro` (français) et `pages/en/index.astro` (anglais), textes dans `lib/i18n.ts` .
- **Données** : avant la compilation, `scripts/sync-data.mjs` copie `data/results/public/results.json` dans le site. Sans résultats, le site affiche « résultats à venir » : aucun chiffre n'est inventé.
- **Couleurs** : palette testée pour le daltonisme ; chaque modèle garde sa couleur, quel que soit son rang.
- **Partage** : `scripts/visuals/generate.py` produit les images PNG pour LinkedIn et l'aperçu des liens (Open Graph).

Adresse : `hasdaouiryma16-cmd.github.io/kabyle-llm-benchmark`

16 Organisation du dépôt

```
kabyle-llm-benchmark/
├── backend/                  l'API Python
│   ├── app/                  le code (chapitre 5)
│   │   └── api/              les routes HTTP par thème
│   ├── config/models.yaml    la liste des modèles (chapitre 7)
│   ├── migrations/           évolutions du schéma de la base (Alembic)
│   ├── tests/                tests automatiques (pytest)
│   ├── var/                  base lab.db et sauvegardes · PAS dans Git
│   ├── Dockerfile            recette de l'image Docker de l'API
│   └── pyproject.toml        dépendances Python (gérées avec uv)
├── lab-ui/                   l'interface du labo (React, chapitre 6)
├── site/                     le site public (Astro, chapitre 15)
├── data/
│   ├── test/                 le jeu de test public + MANIFEST.json
│   │   └── private/          tests privés · PAS dans Git
│   ├── prompts/              les consignes du mode guidé
│   ├── train/                données d'amélioration (seul STATS.json est dans Git)
│   └── results/public/       résultats exportés pour le site
├── pipelines/                téléchargement, préparation, contrôle, index RAG
├── notebooks/                fine-tuning QLoRA sur Colab
├── scripts/                  sauvegarde, images de partage
├── docs/                     documentation (méthodologie, métriques, décisions…)
├── .github/workflows/        CI, CodeQL, déploiement du site
├── docker-compose.yml        les 3 services du labo
├── .env.example              modèle du fichier de clés (sans valeurs)
└── README.md / README.en.md  présentation du projet
```

La documentation existante, à lire ensuite

Document	Contenu
`docs/methodology.md`	Le protocole scientifique complet
`docs/metrics.md`	Le détail des métriques
`docs/lab.md`	Le mode d'emploi quotidien du labo
`docs/phase2.md`	Le plan d'amélioration du petit modèle
`docs/decisions.md`	Les 35 décisions techniques et leurs raisons
`docs/add-a-model.md`	Ajouter un modèle ou un fournisseur
`data/train/SOURCES.md`	Les corpus, leurs licences, ceux qui sont exclus et pourquoi

Les technologies, en résumé

Domaine	Outils
Backend	Python, FastAPI, SQLModel, SQLite, Alembic, httpx, sacrebleu, numpy, uv, ruff, pytest

Interface	React, TypeScript, Vite, Tailwind CSS, Vitest, ESLint
Site	Astro, React, Recharts
IA locale	Ollama, Qwen3.5 4B, bge-m3 ; Unsloth (QLoRA) sur Colab
Infra et DevOps	Docker, Docker Compose, GitHub Actions, GitHub Pages, gitleaks, CodeQL, Dependabot

17 Où en est le projet, et la suite

État au 31 octobre 2026

Élément	État
Labo (API, interface, base, Docker)	terminé
Jeu de test v1.0.0 : 52 tests, publié (41 publics)	terminé
Données d'amélioration préparées et décontaminées	terminé
Sécurité : audit, protections, 0 alerte	terminé
CI/CD et site public (en attente de résultats)	terminé
API gratuites (Gemini, Mistral, OpenRouter) avec verrou tests publics	terminé
Ollama dans Docker, Qwen3.5 4B et bge-m3 téléchargés	terminé
Clés d'API dans `.env`	à faire (toi)
Index RAG	bloqué : mémoire
Premier benchmark, notation, résultats	à faire
Fine-tuning QLoRA sur Colab	à faire
Publication des résultats et visuels LinkedIn	à faire

Les prochaines étapes, dans l'ordre

1. **Créer une clé gratuite Gemini** (aistudio.google.com) et la coller dans `.env`, puis `docker compose restart api`.
2. **Premier petit run** en mode test (2 questions par catégorie, partie publique) pour vérifier que tout fonctionne.
3. **Run complet** sur les tests publics avec les modèles gratuits ; ajouter les modèles payants plus tard si tu le souhaites.
4. **Noter à l'aveugle** toutes les réponses.
5. **Construire l'index RAG** (moins de paires, ou sur Colab), puis mesurer Qwen de base, few-shot et RAG.
6. **Fine-tuning QLoRA** sur Colab, puis mesurer la variante fine-tunée.
7. **Exporter** les résultats, générer les visuels, publier.

Les limites assumées

- **Une seule annotatrice** : les notes reflètent ta variété de kabyle et tes choix d'orthographe. Plusieurs références par test atténuent ce biais.
- **Un jeu de taille modeste** : les intervalles de confiance seront larges ; les petits écarts ne seront pas significatifs.

- **chrF++ ne mesure qu'une ressemblance de surface** : la note humaine fait foi.
- **Des résultats datés** : ils valent pour les versions de modèles et la date indiquées.
- **Ton PC** (8 Go de RAM, sans GPU NVIDIA) limite les modèles locaux : les étapes lourdes passent par Colab.

18 Glossaire et commandes utiles

API
Porte d'entrée par laquelle un programme en appelle un autre (ici : le labo appelle les fournisseurs ; l'interface appelle le backend).

Benchmark
Examen standardisé pour comparer des modèles.

Bootstrap
Méthode qui estime la fiabilité d'une moyenne en la recalculant sur 1 000 tirages au hasard.

chrF++
Score de 0 à 100 qui mesure la ressemblance entre deux textes à partir de morceaux de caractères.

CI/CD
Vérifications automatiques à chaque push, et déploiement automatique si tout est vert.

Conteneur
Petite machine isolée qui contient un programme et tout ce dont il a besoin (Docker).

Contamination
Quand un modèle a vu les questions du test pendant son entraînement ou dans son aide.

Embedding
Liste de nombres qui représente le sens d'une phrase.

Empreinte (SHA-256)
Code calculé à partir d'un contenu ; il change dès que le contenu change.

Few-shot
Donner quelques exemples résolus au modèle juste avant la question.

Fine-tuning
Continuer l'entraînement d'un modèle sur des données spécifiques.

GGUF
Format de fichier d'un modèle quantifié, utilisé par Ollama.

GPU
Carte graphique ; fait les calculs des modèles beaucoup plus vite qu'un processeur.

Inférence
Le fait de faire répondre un modèle déjà entraîné.

JSON / JSONL
Format texte structuré ; JSONL = un objet JSON par ligne.

LLM
Grand modèle de langage : programme qui génère du texte en devinant le mot suivant.

LoRA / QLoRA
Fine-tuning léger qui entraîne une petite « rustine » au lieu du modèle entier ; QLoRA le fait sur un modèle compressé en 4 bits.

Ollama
Logiciel qui fait tourner des modèles open source sur sa propre machine.

Paramètres
Les nombres appris par un modèle ; « 4B » = 4 milliards.

Prompt
Le texte envoyé au modèle.

Quantification
Compression d'un modèle en arrondissant ses paramètres.

RAG
Chercher des textes utiles dans une bibliothèque et les donner au modèle avec la question.

Température
Niveau de hasard dans les réponses ; 0 = réponses stables.

Token
Morceau de mot ; unité de lecture et de facturation des modèles.

Commandes utiles

```
# Lancer et arrêter le labo
docker compose up -d                          # API + interface
docker compose --profile ollama up -d         # + modèles locaux
docker compose down                           # arrêter (les données restent)
docker compose restart api                    # après une modification de .env

# Données de la phase 2
docker compose run --rm api python -m pipelines.prepare
docker compose run --rm api python -m pipelines.check_contamination
docker compose run --rm api python -m pipelines.build_rag_index --limit 5000

# Sauvegarde (à faire régulièrement)
docker compose run --rm api python -m scripts.backup

# Tests automatiques
docker compose run --rm api pytest
docker compose run --rm lab-ui npm test

# Adresses
Interface du labo     http://localhost:5173
API et sa doc         http://127.0.0.1:8800/docs
```